PROJECT

Of

R PROGRAMMING LAB

SUBJECT CODE: 25CAP-161

Submitted By:

Ayushi Sharma [25BCD10184]

Sunidhi Sharma[25BCD10021]

Submitted To:

Ms. Sushma Ma'am

TITLE - TWITTER SENTIMENT DASHBOARD

University Institute of Computing

Chandigarh University, Gharuan, Mohali

INDEX

Sr. No.	Topic	Page
1	Introduction About the Topic	3
2	Objectives	3
3	Data Description	4
4	Tools and Technologies	4
4.1	Code with Explanation – UI (app.R)	5
4.2	Code with Explanation – Server Logic (app.R)	7
4.3	Code with Explanation – Helper Functions	10
5	Output / Screenshots	11
6	Conclusion	12
7	Learning Outcomes	12
8	GitHub Link & README	13

1. Introduction About the Topic

Sentiment analysis, also known as opinion mining, is a branch of Natural Language Processing (NLP) that computationally identifies and categorises opinions expressed in text. With social media platforms like Twitter generating billions of posts daily, automated sentiment monitoring has become a critical capability for businesses, governments, and researchers.

The Twitter Sentiment Studio is an interactive web application built with R Shiny that simulates a live stream of tweets, scores each tweet using the syuzhet NLP package, and presents the results through a clean dashboard featuring animated KPI cards, an arc-based sentiment gauge, a live trend chart, and a colour-coded tweet feed — all refreshing automatically every two seconds.

Because Twitter/X now requires paid API access, the project uses a curated tweet pool that can be replaced with a real API call, making the architecture production-ready.

2. Objectives

- Build a real-time tweet sentiment dashboard in R Shiny with no external API dependency.
- Apply the syuzhet NLP scoring method to classify tweets as Positive, Negative, or Neutral.
- Provide interactive controls — Start, Stop, and Reset — to manage the live tweet stream.
- Visualise sentiment data through an arc-based gauge plot and a running trend line chart.
- Display a colour-coded live tweet feed showing timestamp, text, and sentiment score.
- Demonstrate reactive Shiny patterns: reactiveVal, invalidateLater, observeEvent, isolate.
- Produce a clean, professional UI via inline CSS and the shinyjs package for button-state management.

3. Data Description

All tweet data is generated in-memory at runtime by the generate_tweet() function. The pool of 10 sample tweets covers strongly positive, mildly positive, neutral, mildly negative, and strongly negative sentiments, ensuring all dashboard visual states are demonstrated.

Parameter	Details
Source	In-app simulated tweet pool (10 template tweets)
Generation Rate	1 tweet every 2 seconds via invalidateLater(2000)
Scoring Method	syuzhet package — get_sentiment(text, method = 'syuzhet')
Score Range	Continuous float; typically -1.0 to +1.0
Thresholds	Positive: score > 0.2 | Negative: score < -0.2 | Neutral: otherwise
Buffer Size	Last 30 tweets retained in the reactive data frame
Tweet Fields	time (HH:MM:SS), text (tweet content), score (sentiment float)
Display Order	Newest tweet at the top (data frame reversed before render)

Tweet Pool (verbatim from code)

- "I love this product! Amazing!"
- "This is terrible, worst experience"
- "Pretty good, not bad"
- "Feeling neutral about this"
- "Wow! Incredible work!"
- "Disappointed, waste of money"
- "Could be better"
- "I hate this so annoying"
- "Best day ever! So happy!"
- "This makes me angry"

4. Tools and Technologies

Package / Tool	Version	Role in Project
R	4.3+	Core programming language
RStudio	2023.12+	IDE for development and debugging
shiny	1.8+	Reactive web application framework
shinyjs	2.1+	Enable / disable buttons via JavaScript from R
syuzhet	1.0.7	NLP sentiment scoring (Jockers 2017 lexicon)
Base R graphics	built-in	Arc gauge and trend chart inside renderPlot()
HTML / CSS	—	Custom inline styles: cards, buttons, tweet rows

4.1 Code with Explanation – UI Definition (app.R)

The UI is constructed with fluidPage(). Custom CSS is injected via tags$style(HTML(...)). The layout has five visual layers: header banner, four KPI stat cards, two chart panels (gauge + trend), live tweet feed, and the control button row.

```
# -- Library Imports ----------------------------------------------
library(shiny)    # Core Shiny framework
library(shinyjs)  # JS helpers: enable / disable buttons
library(syuzhet)  # Sentiment scoring (syuzhet lexicon)

# -- UI ------------------------------------------------------------
ui <- fluidPage(
  useShinyjs(),   # Must be called inside the UI to initialise shinyjs

  # -- Inline CSS ---------------------------------------------------
  tags$style(HTML(
    "body { background: #faf8f5; font-family: Arial; }

     # Header card
     .header { text-align: center; padding: 30px; background: white;
               border-radius: 20px; margin-bottom: 20px; }

     # KPI card row (CSS flexbox)
     .stats { display: flex; gap: 20px; margin-bottom: 20px; }
     .stat  { background: white; padding: 20px; border-radius: 15px;
              text-align: center; flex: 1; }
     .stat h2 { margin: 10px 0; font-size: 2em; }

     # Chart row (CSS flexbox)
     .charts { display: flex; gap: 20px; margin-bottom: 20px; }
     .chart  { background: white; padding: 20px; border-radius: 15px; flex: 1; }

     # Tweet feed and button section
     .feed    { background: white; padding: 20px; border-radius: 15px;
                margin-bottom: 20px; }
     .buttons { text-align: center; margin-top: 20px; }

     # Shared button style
     button { padding: 10px 20px; margin: 0 10px; border: none;
              border-radius: 25px; cursor: pointer; font-weight: bold; }
     .btn-start { background: #4caf50; color: white; }
     .btn-stop  { background: #f44336; color: white; }
     .btn-reset { background: #9e9e9e; color: white; }

     # Tweet row colours
     .tweet-row { border-bottom: 1px solid #eee; padding: 10px; }
     .positive  { color: #4caf50; }
     .negative  { color: #f44336; }
     .neutral   { color: #9e9e9e; }"
  )),

  # -- Page layout ---------------------------------------------------
  div(style = 'max-width: 1200px; margin: 0 auto; padding: 20px;',

    # Banner
    div(class = 'header',
      h1('Twitter Sentiment Studio'),
      p('Real-time mood analytics')
    ),

    # KPI Cards
    div(class = 'stats',
      div(class='stat', h4('Total Tweets'), h2(textOutput('total'))),
```

```
      div(class='stat', h4('Avg Score'),    h2(textOutput('avg'))),
      div(class='stat', h4('Mood'),         h2(textOutput('mood'))),
      div(class='stat', h4('Status'),       h2(textOutput('status')))
    ),

    # Charts
    div(class = 'charts',
      div(class='chart', h4('Sentiment Gauge'), plotOutput('gauge', height='150px')),
      div(class='chart', h4('Trend'),           plotOutput('trend', height='150px'))
    ),

    # Live tweet feed
    div(class = 'feed',
      h4('Live Feed'),
      uiOutput('tweets_list')
    ),

    # Control buttons
    div(class = 'buttons',
      actionButton('start', 'START', class = 'btn-start'),
      actionButton('stop',  'STOP',  class = 'btn-stop'),
      actionButton('reset', 'RESET', class = 'btn-reset')
    )
  )
)
```

4.2 Code with Explanation – Server Logic (app.R)

The server function manages the reactive data store, three button event handlers, the auto-tweet observer, and five reactive output renderers. Button states are managed via shinyjs.

```
server <- function(input, output, session) {

  # -- Reactive data store ----------------------------------------
  tweet_data <- reactiveVal(data.frame(
    time  = character(),
    text  = character(),
    score = numeric(),
    stringsAsFactors = FALSE
  ))

  running <- reactiveVal(FALSE)   # TRUE = stream is active

  # -- Internal: append one tweet ----------------------------------
  add_tweet <- function() {
    text     <- generate_tweet()
    score    <- get_score(text)
    new_row  <- data.frame(
      time  = format(Sys.time(), '%H:%M:%S'),
      text  = text,
      score = score,
      stringsAsFactors = FALSE
    )
    combined <- rbind(tweet_data(), new_row)
    if (nrow(combined) > 30) combined <- tail(combined, 30)
    tweet_data(combined)
  }

  # -- START button ------------------------------------------------
  observeEvent(input$start, {
    running(TRUE)
    shinyjs::disable('start')
    shinyjs::enable('stop')
  })

  # -- STOP button -------------------------------------------------
  observeEvent(input$stop, {
    running(FALSE)
    shinyjs::enable('start')
    shinyjs::disable('stop')
  })

  # -- RESET button ------------------------------------------------
  observeEvent(input$reset, {
    tweet_data(data.frame(time=character(), text=character(),
                          score=numeric(), stringsAsFactors=FALSE))
    running(FALSE)
    shinyjs::enable('start')
    shinyjs::disable('stop')
  })

  # -- Auto-tweet observer ----------------------------------------
  # invalidateLater(2000) re-executes every 2000 ms.
  # isolate() prevents recursive triggering.
  observe({
    invalidateLater(2000)
    if (running()) { isolate(add_tweet()) }
  })

  # -- KPI outputs -------------------------------------------------
  output$total <- renderText({ nrow(tweet_data()) })
```

```
output$avg <- renderText({
  data <- tweet data()
  if (nrow(data) == 0) return('0.00')
  sprintf('%.2f', mean(data$score))
})

output$mood <- renderText({
  data <- tweet data()
  if (nrow(data) == 0) return('---')
  avg <- mean(data$score)
  if (avg >  0.2) return('Positive')
  if (avg < -0.2) return('Negative')
  return('Neutral')
})

output$status <- renderText({
  if (running()) return('LIVE') else return('PAUSED')
})

# -- Gauge chart ---------------------------------------------------
output$gauge <- renderPlot({
  data <- tweet data()
  avg  <- if (nrow(data) > 0) mean(data$score) else 0
  par(bg='white', mar=c(1,1,1,1))
  plot(0, 0, type='n', xlim=c(-1.2,1.2), ylim=c(-1.2,1.2), axes=FALSE)
  for (i in seq(-0.9, 0.9, length.out=11)) {
    color <- if(i < -0.3) '#ffcdd2'
             else if(i < 0.3) '#fff9c4'
             else '#c8e6c9'
    lines(c(i, i*0.8),
          c(sqrt(1-i^2)*0.8, sqrt(1-(i*0.8)^2)*0.8),
          col=color, lwd=4)
  }
  angle <- avg * pi / 2
  arrows(0, 0, cos(angle)*0.7, sin(angle)*0.7,
         col = if(avg > 0) '#4caf50' else '#f44336', lwd=2)
  points(0, 0, pch=19)
  text(0, -0.5, paste('Score:', round(avg, 2)), col='#666', cex=0.9)
})

# -- Trend chart ---------------------------------------------------
output$trend <- renderPlot({
  data <- tweet data()
  if (nrow(data) == 0) {
    plot(0, 0, type='n', main='No data yet', axes=FALSE)
    text(0, 0, 'Click START to begin', col='#999')
    return()
  }
  par(bg='white', mar=c(2,2,1,1))
  plot(1:nrow(data), data$score, type='b',
       xlab='', ylab='', ylim=c(-1,1), col='#4caf50', pch=19)
  abline(h=0, col='#f44336', lty=2)
  grid(col='#eee')
})

# -- Live tweet feed -----------------------------------------------
output$tweets_list <- renderUI({
  data <- tweet data()
  if (nrow(data) == 0)
    return(div(class='tweet-row', 'Click START to see tweets'))
  data <- data[rev(1:nrow(data)), ]
  tweet_divs <- lapply(1:nrow(data), function(i) {
    score class <- if(data$score[i] > 0) 'positive'
                   else if(data$score[i] < 0) 'negative'
                   else 'neutral'
    div(class = 'tweet-row',
      span(data$time[i], style='color:#999; margin-right:15px;'),
      span(data$text[i]),
```

```
        span(class=score class, style='float:right;',
             paste('Score:', data$score[i]))
      )
    })
    do.call(tagList, tweet_divs)
  })
}

shinyApp(ui = ui, server = server)
```

4.3 Code with Explanation – Helper Functions

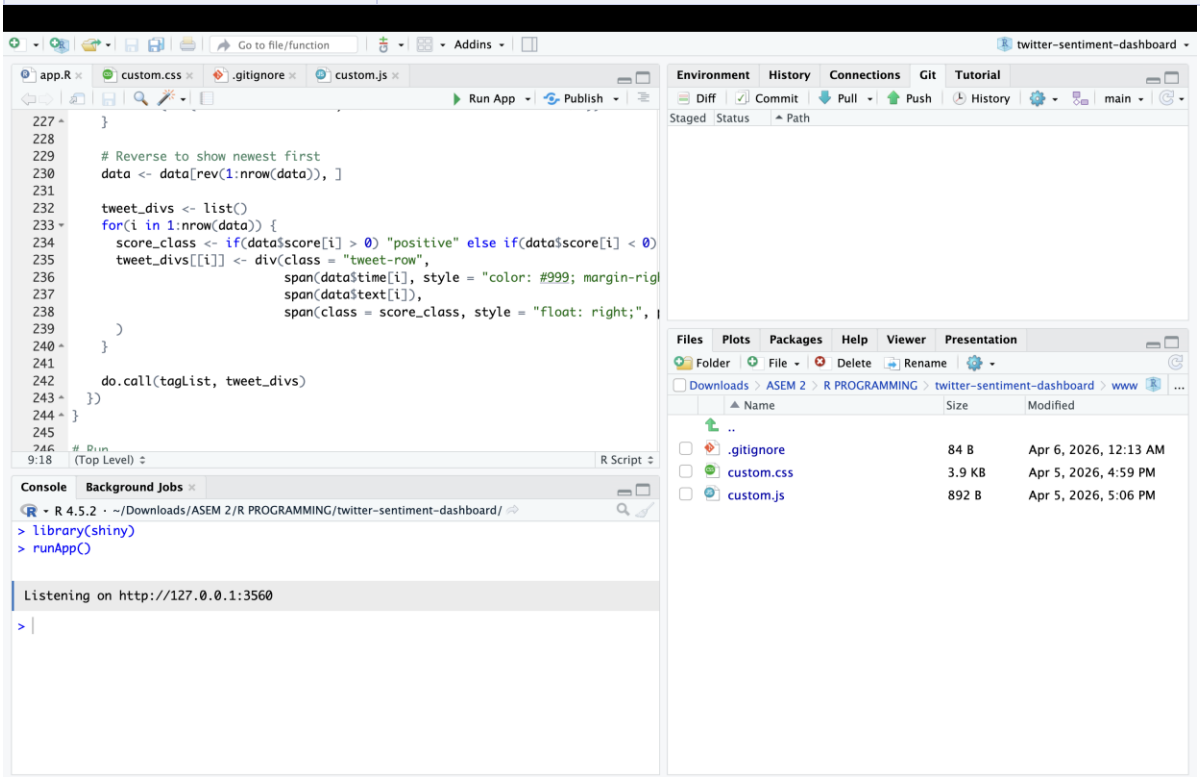
Three utility functions defined at the top of app.R isolate business logic from UI and server layers, improving readability and testability.

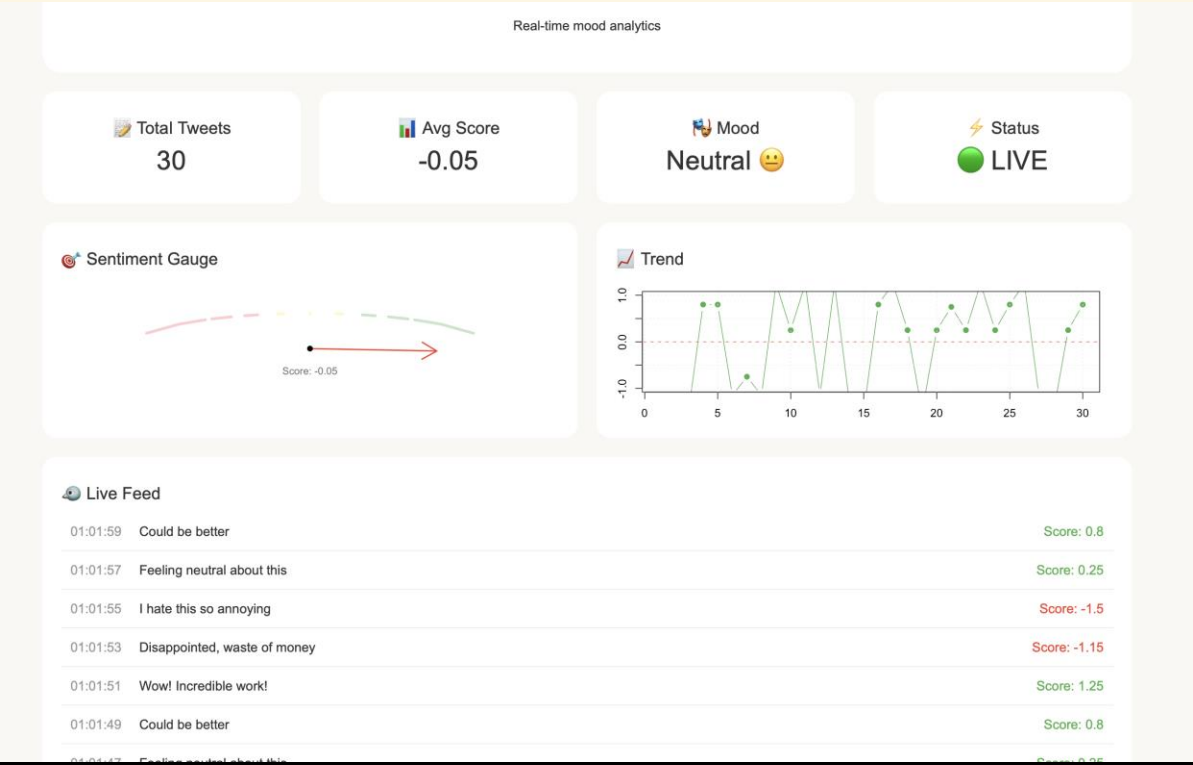
```
# -- Helper 1: Tweet Generator ----------------------------------------
# Randomly selects one tweet from a 10-item pool (uniform probability).
generate_tweet <- function() {
  tweets <- c(
    "I love this product! Amazing!",
    "This is terrible, worst experience",
    "Pretty good, not bad",
    "Feeling neutral about this",
    "Wow! Incredible work!",
    "Disappointed, waste of money",
    "Could be better",
    "I hate this so annoying",
    "Best day ever! So happy!",
    "This makes me angry"
  )
  return(sample(tweets, 1))
}

# -- Helper 2: Sentiment Scorer ---------------------------------------
# Applies the syuzhet method (Jockers 2017 lexicon).
get_score <- function(text) {
  score <- get_sentiment(text, method = 'syuzhet')
  return(round(score, 2))
}

# -- Helper 3: Label Classifier ---------------------------------------
# Maps numeric score to a human-readable label using fixed thresholds.
get_label <- function(score) {
  if (score >  0.2) return('Positive')
  if (score < -0.2) return('Negative')
  return('Neutral')
}
```

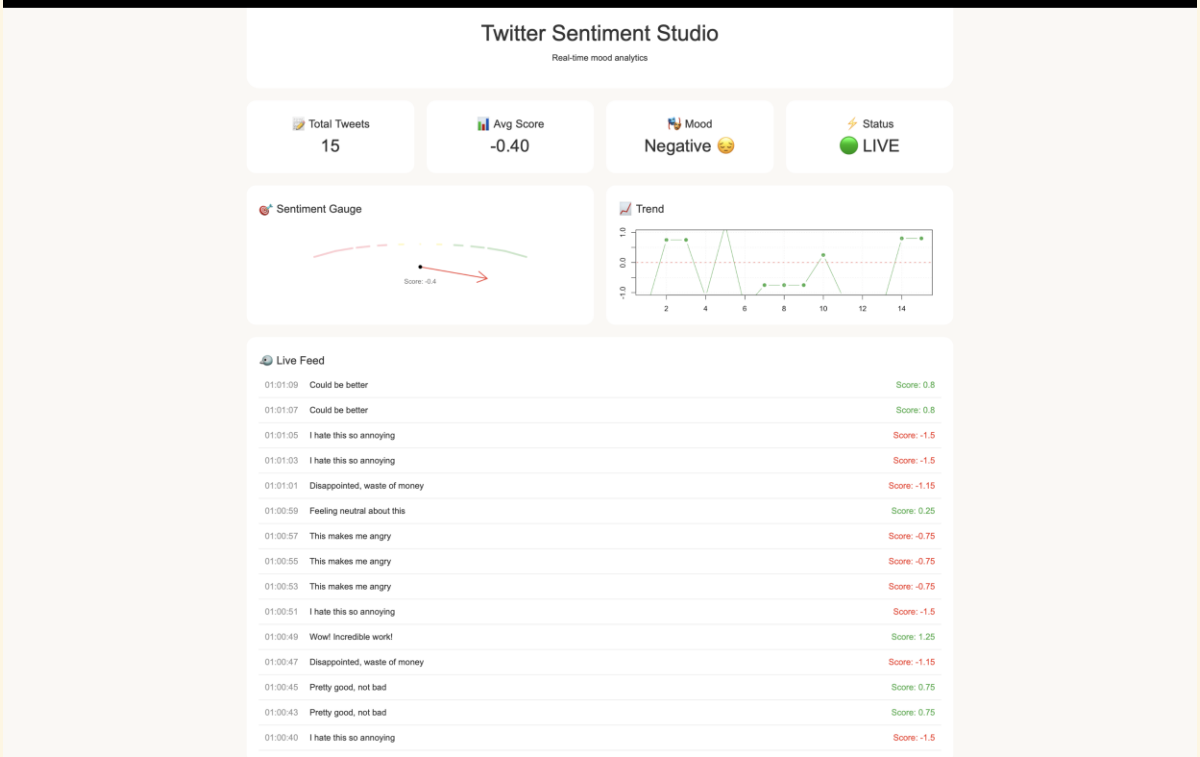
5. Output / Screenshots

After launching app.R and clicking START, the following components are visible and refresh automatically every 2 seconds:

Component	Description
Header Banner	'Twitter Sentiment Studio' title on white rounded card
KPI – Total	Running count of tweets processed in the current session
KPI – Avg Score	Rolling mean of all syuzhet scores, 2-decimal format
KPI – Mood	Text label: Positive / Negative / Neutral
KPI – Status	'LIVE' when streaming; 'PAUSED' when stopped
Gauge (left)	Arc chart: red / yellow / green zones; needle at average score
Trend (right)	Line chart of all stored scores; red dashed zero-reference
Live Tweet Feed	Newest tweet at top; timestamp grey; score in green/red/grey
START button	Activates stream; disables itself via shinyjs::disable()
STOP button	Pauses stream; re-enables START
RESET button	Clears all data and restores initial state




Real-time mood analytics
Total Tweets
30
Avg Score
-0.05
Mood
Neutral
Status
LIVE
Sentiment Gauge
Score: -0.05
Trend
1.0
0.0
-1.0
0
5
10
15
20
25
30
Live Feed
01:01:59 Could be better Score: 0.8
01:01:57 Feeling neutral about this Score: 0.25
01:01:55 I hate this so annoying Score: -1.5
01:01:53 Disappointed, waste of money Score: -1.15
01:01:51 Wow! Incredible work! Score: 1.25
01:01:49 Could be better Score: 0.8


Twitter Sentiment Studio
Real-time mood analytics
Total Tweets
15
Avg Score
-0.40
Mood
Negative
Status
LIVE
Sentiment Gauge
Score: -0.4
Trend
1.0
0.0
-1.0
2
4
6
8
10
12
14
Live Feed
01:01:09 Could be better Score: 0.8
01:01:07 Could be better Score: 0.8
01:01:05 I hate this so annoying Score: -1.5
01:01:03 I hate this so annoying Score: -1.5
01:01:01 Disappointed, waste of money Score: -1.15
01:00:59 Feeling neutral about this Score: 0.25
01:00:57 This makes me angry Score: -0.75
01:00:55 This makes me angry Score: -0.75
01:00:53 This makes me angry Score: -0.75
01:00:51 I hate this so annoying Score: -1.5
01:00:49 Wow! Incredible work! Score: 1.25
01:00:47 Disappointed, waste of money Score: -1.15
01:00:45 Pretty good, not bad Score: 0.75
01:00:43 Pretty good, not bad Score: 0.75
01:00:40 I hate this so annoying Score: -1.5

6. Conclusion

The Twitter Sentiment Studio successfully delivers a self-contained, real-time sentiment analysis web application built entirely in R. The project meets all stated objectives:

- A tweet simulator provides a diverse-sentiment stream without requiring external API credentials.
- The syuzhet NLP lexicon scores each tweet in real time, producing a continuous value classified into three categories.
- A fully reactive Shiny architecture drives all UI updates without page reloads or manual refresh.
- Custom inline CSS and shinyjs deliver a professional interface with proper button-state transitions.
- The modular helper design makes it trivial to replace the simulator with a live Twitter/X API call.

7. Learning Outcomes

- Applied NLP sentiment analysis using the syuzhet lexicon to classify social media text.
- Mastered Shiny reactive programming: reactiveVal(), observe(), observeEvent(), invalidateLater(), isolate().
- Built dynamic HTML UI from the server using renderUI() and tagList().
- Used base R graphics (par(), arrows(), abline(), grid()) inside renderPlot() for custom charts.
- Managed button state across the session with shinyjs::enable() and shinyjs::disable().
- Implemented a rolling data window (last 30 tweets) using tail() on a reactive data frame.
- Practised software documentation: structured README, inline code comments, and academic report writing.

8. GitHub Link & README Setup Instructions

GitHub Repository: https://github.com/ayushixx/twitter-sentiment-dashboard

README.md

```
# Twitter Sentiment Studio
# Real-Time Mood Analytics  |  R Shiny + syuzhet NLP
# Chandigarh University - Mini Project 2024-25

## Project Structure
  twitter-sentiment-studio/
  |-- app.R          <- Single-file Shiny app (UI + Server + Helpers)
  |-- README.md

## Step 1 - Install Packages  (run once in R console)
  install.packages(c("shiny", "shinyjs", "syuzhet"))

## Step 2 - Clone
  git clone https://github.com/ayushixx/twitter-sentiment-studio.git

## Step 3 - Run
  library(shiny); runApp(".", launch.browser = TRUE)

## Step 4 - Using the Dashboard
  1. Click START   — tweets stream every 2 seconds
  2. Click STOP    — pauses the stream (data retained)
  3. Click RESET   — clears all data
```